Day 22 (Wed Nov 11): Narrow except, else, grade_counts, input loop

Smooth | Python | covers only what was really done

Key ideas

- A narrow except (except ZeroDivisionError) catches only that error. A TypeError is not caught, leaves the function and stops the program with a traceback.
- ZeroDivisionError covers 10 / 0, 10 / 0.0, 10 % 0 and 10 // 0.
- try/except/else: else runs only when the try finishes with no error, and is skipped when except runs.
- An error raised inside else is NOT caught by the except above it, so a bug there stays visible.
- Keep the try block to the risky line; code that depends on its success goes in else.
- Dict counting engine: first-time step (grade not yet a key) and update (+1) for every item. The helper grade_of(mark) is called inside the loop, once per mark.
- Input loop: while True, int(input(...)) is the line that can raise ValueError, return ends the loop and the function.

Examples (run and verified)

```
try:
    n = int(s)
except ValueError:
    print("A")
else:
    print("B")
print("C")
# s = "42" prints B then C;  s = "x" prints A then C

int(" 7 ")  -> 7        int("-3") -> -3        int("12.5"), int("") raise ValueError

def grade_counts(marks):
    counts = {}
    for mark in marks:
        grade = grade_of(mark)
        ...first-time step, then update...
    return counts
# grade_counts([80, 74.5, 49, 90]) -> {'A': 2, 'B': 1, 'C': 1};  grade_counts([]) -> {}
```

Interview edge

- except Exception is almost a bare except: it hides the TypeError and returns None, so the caller cannot tell "b was 0" from "b was a string".
- In an if/elif chain use >= lower bounds; "<= 74" leaves a gap, so 74.5 falls to else.
- Shorter counting forms exist: dict.get(key, 0) + 1 and collections.Counter. Learn the long form first.

My mistakes and the fix

- Wrote except Exception where the spec said only b = 0. Fix: name the exact error the spec asks for.
- Said ValueError for safe_divide(10, "a"). It raises TypeError (wrong type for /). ValueError is right type, bad content.
- Described the first-time step as "if the dict is empty". Fix: it runs when the grade is not yet a key.
- Answered for my old code, not the version asked. Fix: re-read the question and the code first.
- Vague reasons ("cannot convert string to int"). Fix: name the type and the content.
- Predictions not written before running. Fix: write the prediction as a comment first.

How I improved

- safe_divide retest passed from a blank file with the correct narrow except and correct reasons for all three calls.
- grade_counts with the grade_of helper passed, including boundaries and the empty list.
- ask_number worked in both versions (return in try, return in else); I rewrote it into else without help.

Practice to retry (no solutions)

- From a blank file write safe_divide(a, b): narrow except, no print inside. Give each result of (10, 2), (10, 0), (10, "a") and say returns or raises.
- Rewrite grade_of with elif written as mark <= 74. Trace 75, 74.5, 74, 74.0001 through every check.
- Write to_int(text) that returns the integer or None. Which error do you catch? What do "7", "7.5", "" give?
- Which error does each raise: int("abc"), int(None), 5 + "5", 5 / 0, [1, 2][5]? Check by running.

Recall questions (from memory)

- Why does safe_divide(10, "a") raise with a narrow except, and which error?
- When does else run, and is an error inside else caught by the except above?
- Which line in ask_number raises ValueError, and what ends the loop?
- Where is the helper called in grade_counts, and when does the first-time step run?

Open items

- Descriptive statistics, Text and date functions, Marks analyzer.
- Git conflict redo from a blank folder; README and .gitignore for one project.